



FRONT-END E MOBILE DEVELOPMENT EM SISTEMAS DE IA

GLOBAL SOLUTION 2026/1

1 Contexto e Motivação

Eventos climáticos extremos, queimadas, enchentes, secas prolongadas e decisões agrícolas em tempo real exigem ferramentas que transformem volumes massivos de dados — satelitais, meteorológicos e geolocalizados — em informação acionável para tomadores de decisão não-técnicos. Um modelo de IA, por mais preciso que seja, é inútil se a interface que o expõe ao usuário final for confusa, lenta ou pouco confiável.

A disciplina de Front-end em Sistemas de IA preparou o aluno até o momento para projetar essa camada de interface: organização arquitetural em camadas, controle do ciclo de execução de Streamlit/Gradio, componentização, cache, gráficos interativos com Matplotlib e Plotly, e princípios de design (feedback humano, design para latência, cores semânticas, organização de layout). Esta Global Solution é o momento de aplicar esse repertório em um problema real.

2 O Desafio

A atividade poderá ser realizada **individualmente, em dupla ou em trio**;

Cada grupo deve projetar e implementar um **dashboard interativo** que consuma dados **satelitais, climáticos ou de geolocalização** (que podem ser simulados/mockados), apresentando esses dados de forma que facilite a tomada de decisão em um problema concreto de domínio — agricultura, desastres ambientais, logística climática, monitoramento de queimadas, qualidade do ar, gestão hídrica, ou outro que o grupo defenda no pitch.

A Global Solution é uma competição. Entreguem a Prova de Conceito (POC) funcionando com os requisitos da disciplina, mas não tenham medo de **extrapolar na criatividade** da solução para se destacarem entre as turmas!

3 Requisitos Técnicos Obrigatórios

Para demonstrar o domínio dos conceitos ministrados em sala, sua aplicação deve obrigatoriamente contemplar os seguintes itens:

3.1 Ciclo de execução e estado

Escolher um framework para trabalhar: **StreamLit** ou **Gradio**. O grupo deve demonstrar domínio do ciclo de execução do framework escolhido:

Streamlit: uso correto de `st.session_state`, `@st.cache_data` e/ou `@st.cache_resource` para evitar re-execução de chamadas custosas a cada rerun.

Gradio: uso correto de `gr.State`, listeners `.click()` / `.change()` / `.then()`, e separação clara entre componentes reativos e estado persistente.

3.2 Arquitetura do Projeto

O código **não pode ser entregue em um único arquivo monolítico**. A solução deve refletir boas práticas de engenharia de software e ser componentizada utilizando a estrutura abordada em aula, ou alguma estrutura que o grupo esteja acostumado com a nomenclatura:

providers/ — acesso a dados externos (chamadas HTTP, autenticação de API, parsing bruto).

pipelines/ — orquestração e transformação de dados (limpeza, agregação, enriquecimento).

features/ — separação das telas em responsabilidades diferentes.

state/ — gerenciamento centralizado de estado (`st.session_state` ou `gr.State`).

ui/ — componentes visuais e composição de layout.

3.3 Arquitetura do Projeto

Pelo menos **3 filtros interativos** que alterem os dados exibidos (ex.: intervalo de datas, região geográfica, categoria, threshold).

Pelo menos **2 visualizações gráficas**, sendo no mínimo uma com Plotly (interativa, com hover e zoom). Matplotlib é aceito como gráfico complementar.

Componentização real: ao menos um componente da UI deve ser função/módulo reutilizável e ser invocado em mais de um lugar ou com diferentes parâmetros (ex.: um gráfico, um componente de sidebar utilizado também em tela, um markdown)

3.4 Teoria do Design e UX/UI

Layout Organizado: Uso inteligente de *tabs* (abas), colunas, linhas e *sidebar* para guiar a jornada do usuário.

Design para Latência: Tratamento visual para o tempo de carregamento dos dados (uso de *spinners*, barras de progresso ou mensagens de *loading*).

Cores Semânticas: Aplicação intencional de cores para indicar criticidade (ex: vermelho para alertas de desastres, verde para áreas seguras/produtivas).

Feedback Humano (Human-in-the-loop): A interface deve possuir ao menos um componente onde o usuário precise interagir com um "insight" gerado pelo sistema (ex: aprovar o envio de um alerta de evacuação sugerido pela análise de dados).

4 Entregáveis

A entrega é feita em **um único repositório público no GitHub** contendo todos os itens abaixo. O link do repositório é submetido no Portal do Aluno junto com o vídeo.

1. Link do Repositório (GitHub): Contendo o código-fonte organizado na arquitetura, o arquivo requirements.txt com as bibliotecas necessárias e um README.md com a descrição do problema, fonte de dados escolhida, justificativa de framework, diagrama de arquitetura, instruções de instalação e execução.

2. Vídeo: Hospedado no YouTube (Não listado) ou em arquivo no github para baixar junto do código.

No vídeo, o grupo deve:

Apresentar o problema que a aplicação resolve (O impacto espacial).

Demonstrar a interface funcionando (Navegação, gráficos, filtros, design de latência).

Explicar o código: Mostrar rapidamente como a arquitetura (UI, Providers, Pipeline, etc.) foi implementada, provando a autoria do projeto.

5 Critérios

A nota máxima será reservada apenas para projetos que cumpram rigorosamente todos os requisitos técnicos, apresentem excelência na execução e extrapolem o básico na criatividade da solução.

Critério	Descrição	Nota Máxima
1. Arquitetura e Estrutura de Código	Código devidamente componentizado nas camadas exigidas (features, UI, Providers, pipeline, state). Projetos entregues em um único arquivo monolítico não terão nota neste critério.	2,5
2. Gerenciamento de Estado e Performance	Uso correto do <i>ciclo de rerun</i> (Streamlit) ou <i>event-driven</i> (Gradio). Aplicação de Cache adequada, evitando recarregamentos desnecessários de dados na tela.	2,0
3. UI/UX e Teoria do Design	Uso correto de colunas, tabs e sidebar. Presença clara de <i>Design para Latência</i> e aplicação correta de <i>Cores Semânticas</i> . Implementação do fluxo de <i>Feedback Humano</i> .	2,5
4. Visualização de Dados	Gráficos interativos (Matplotlib/Plotly) bem formatados, responsivos aos filtros do usuário e com informações relevantes ao tema espacial.	1,5
5. Organização do github e vídeo	Organização dos diretórios, qualidade do README e do requirements.txt	1,5

6 Dicas que podem trazer um diferencial

Não são obrigatórias para composição da nota, mas podem ajudar a terem ideias do que implementar e do que trazer de diferencial para o projeto:

Integração com modelo de IA: o dashboard incorpora um modelo (predição de focos de incêndio, classificação de risco agrícola, detecção de anomalias climáticas) aproveitado de alguma outra matéria, e treinado pelo grupo.

Deploy público funcional: a aplicação estar rodando em Hugging Face Spaces, ou Streamlit Cloud acessível por URL.

Múltiplas fontes integradas: o dashboard cruza dados de duas ou mais origens (ex.: focos de queimada + previsão climática + qualidade do ar).

Design system próprio: tema visual coerente, tipografia intencional, componentes reutilizáveis com identidade visual consistente, um logo e nome de um sistema fictício que trabalha com o contexto que o grupo quer abordar.

Testes automatizados: ao menos a camada de pipelines/features tem testes unitários executáveis, conseguir rodar o pipeline ou provider se tiver, por dentro do visual studio e analisar os prints de console sem precisar rodar a interface inteira. Demonstra que a arquitetura não depende do framework de interface para conseguir rodar.

Storytelling de dados: o dashboard guia o usuário por uma narrativa (do panorama geral ao detalhe), e não apenas exibe gráficos soltos.